To install Gradio from main, run the following command:

```
pip install https://gradio-builds.s3.amazonaws.com/30bf54c187478cdafa70037f088
e353f5629a995/gradio-6.10.0-py3-none-any.whl
```

*Note: Setting in will not work.

← TabbedInterface

Server →

Copy Page

# Blocks

```
with gradio.Blocks():
```

## Description

Blocks is Gradio's low-level API that allows you to create more custom web applications and demos than Interfaces (yet still entirely in Python).

Compared to the Interface class, Blocks offers more flexibility and control over: (1) the layout of components (2) the events that trigger the execution of functions (3) data flows (e.g. inputs can trigger outputs, which can trigger the next level of outputs). Blocks also offers ways to group together related demos such as with tabs.

The basic usage of Blocks is as follows: create a Blocks object, then use it as a context (with the "with" statement), and then define layouts, components, or events within the Blocks context. Finally, call the launch() method to launch the demo.

## Example Usage

```python
import gradio as gr
def update(name):
    return f"Welcome to Gradio, {name}!"

with gr.Blocks() as demo:
    gr.Markdown("Start typing below and then click **Run** to see the output.")
    with gr.Row():
        inp = gr.Textbox(placeholder="What is your name?")
        out = gr.Textbox()
    btn = gr.Button("Run")
    btn.click(fn=update, inputs=inp, outputs=out)

demo.launch()
```

## Initialization

**Parameters**

- `analytics_enabled: bool | None`
- `mode: str`
- `title: str | I18nData`
- `fill_height: bool`
- `fill_width: bool`
- `delete_cache: tuple[int, int] | None`

## Demos

blocks_hello | blocks_flipper | blocks_kinematics

Loading...

## Methods

### launch

```
gradio.Blocks.launch(...)
```

#### Description

Launches a simple web server that serves the demo. Can also be used to create a public link used by anyone to access the demo from their browser by setting share=True.

#### Example Usage

```
import gradio as gr
def reverse(text):
    return text[::-1]
with gr.Blocks() as demo:
    button = gr.Button(value="Reverse")
    button.click(reverse, gr.Textbox(), gr.Textbox())
demo.launch(share=True, auth=("username", "password"))
```

**Parameters**

- `inline: bool | None`
- `inbrowser: bool`
- `share: bool | None`
- `debug: bool`
- `max_threads: int`
- `auth: Callable[[str, str], bool] | tuple[str, str] | list[tuple[str, str]] | None`
- `auth_message: str | None`
- `prevent_thread_lock: bool`

```
show_error: bool ▼
server_name: str | None ▼
server_port: int | None ▼
height: int ▼
width: int | str ▼
favicon_path: str | Path | None ▼
ssl_keyfile: str | None ▼
ssl_certfile: str | None ▼
ssl_keyfile_password: str | None ▼
ssl_verify: bool ▼
quiet: bool ▼
footer_links: list[Literal['api', 'gradio', 'settings'] | dict[str, str]] | None ▼
allowed_paths: list[str] | None ▼
blocked_paths: list[str] | None ▼
root_path: str | None ▼
app_kwargs: dict[str, Any] | None ▼
state_session_capacity: int ▼
share_server_address: str | None ▼
share_server_protocol: Literal['http', 'https'] | None ▼
share_server_tls_certificate: str | None ▼
auth_dependency: Callable[[fastapi.Request], str | None] | None ▼
max_file_size: str | int | None ▼
enable_monitoring: bool | None ▼
strict_cors: bool ▼
node_server_name: str | None ▼
```

node_port: int | None

ssr_mode: bool | None

pwa: bool | None

mcp_server: bool | None

i18n: I18n | None

theme: Theme | str | None

css: str | None

css_paths: str | Path | list[str | Path] | None

js: str | Literal[True] | None

head: str | None

head_paths: str | Path | list[str | Path] | None

## queue

```
gradio.Blocks.queue(...)
```

## Description

By enabling the queue you can control when users know their position in the queue, and set a limit on maximum number of events allowed.

## Example Usage

```
with gr.Blocks() as demo:
    button = gr.Button(label="Generate Image")
    button.click(fn=image_generator, inputs=gr.Textbox(), outputs=gr.Image())
demo.queue(max_size=10)
demo.launch()
```

**Parameters**

status_update_rate: float | Literal['auto']

api_open: bool | None

max_size: int | None

default_concurrency_limit: int | None | Literal['not_set']

## integrate

```
gradio.Blocks.integrate(...)
```

### Description

A catch-all method for integrating with other libraries. This method should be run after launch()

**Parameters**

comet_ml: <class 'inspect._empty'>

wandb: ModuleType | None

mlflow: ModuleType | None

## load

```
gradio.Blocks.load(block, ...)
```

### Description

This listener is triggered when the Blocks initially loads in the browser.

**Parameters**

block: Block | None

fn: Callable | None | Literal['decorator']

inputs: Component | BlockContext | list[Component | BlockContext] | Set[Component |

outputs: Component | BlockContext | list[Component | BlockContext] | Set[Component

api_name: str | None ▼

api_description: str | None | Literal[False] ▼

scroll_to_output: bool ▼

show_progress: Literal['full', 'minimal', 'hidden'] ▼

show_progress_on: Component | list[Component] | None ▼

queue: bool ▼

batch: bool ▼

max_batch_size: int ▼

preprocess: bool ▼

postprocess: bool ▼

cancels: dict[str, Any] | list[dict[str, Any]] | None ▼

trigger_mode: Literal['once', 'multiple', 'always_last'] | None ▼

js: str | Literal[True] | None ▼

concurrency_limit: int | None | Literal['default'] ▼

concurrency_id: str | None ▼

api_visibility: Literal['public', 'private', 'undocumented'] ▼

time_limit: int | None ▼

stream_every: float ▼

key: int | str | tuple[int | str, ...] | None ▼

validator: Callable | None ▼

## unload

```
gradio.Blocks.unload(fn, ...)
```

## Description

This listener is triggered when the user closes or refreshes the tab, ending the user session. It is useful for cleaning up resources when the app is closed.

## Example Usage

```
import gradio as gr
with gr.Blocks() as demo:
    gr.Markdown("# When you close the tab, hello will be printed to the console")
    demo.unload(lambda: print("hello"))
demo.launch()
```

**Parameters**

`fn: Callable[..., Any]`

← TabbedInterface

Server →